UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024
Last modified 2024-3-24.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Spring 2024 semester. Lecture videos are available on YouTube.

Previous Chapter | Table of contents | Next Chapter

CHAPTER 17
Planning and Control through Contact

So far we have developed a fairly strong toolbox for planning and control with "smooth" systems -- systems where the equations of motion are described by a function $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ which is smooth everywhere. But our discussion of the simple models of legged robots illustrated that the dynamics of making and breaking contact with the world are more complex -- these are often modeled as hybrid dynamics with impact discontinuities at the collision event and constrained dynamics during contact (with either soft or hard constraints).

My goal for this chapter is to extend our computational tools into this richer class of models. Many of our core tools still work: trajectory optimization, Lyapunov analysis (e.g. with sums-of-squares), and LQR all have natural equivalents.

Let's start with a warm-up exercise: trajectory optimization for the rimless wheel. We already have basically everything that we need for this, and it will form a nice basis for generalizing our approach throughout the chapter.

Example 17.1 (Trajectory optmization for the rimless wheel)

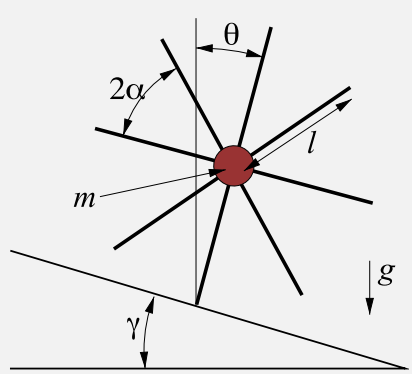

Figure 17.1 - The rimless wheel. The orientation of the stance leg, θ, is measured

clockwise from the vertical axis.

The rimless wheel was our simplest example of a passive-dynamic walker; it has no control inputs but exhibits a passively stable rolling fixed point. We've also already seen that trajectory optimization can be used as a tool for finding limit cycles of a smooth passive system, e.g. by formulating a direct collocation problem:

$$\begin{aligned}\text{find}_{\mathbf{x}[\cdot],h} \quad \text{subject to} \quad & \text{collocation} \quad \text{constraints}(\mathbf{x}[n], \mathbf{x}[n+1], h), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}[N], \\ & h_{min} \leq h \leq h_{max},\end{aligned}$$

where h was the time step between the trajectory break points.

It turns out that applying this to the rimless wheel is quite straight forward. We still want to find a periodic trajectory, but now have to take into account the collision event. We can do this by modifying the periodicity condition. Let's force the initial state to be just after the collision, and the final state to be just before the collision, and make sure they are related to each other via the collision equation:

$$\begin{aligned}\text{find}_{\mathbf{x}[\cdot],h} \quad \text{subject to} \quad & \text{collocation} \quad \text{constraints}(\mathbf{x}[n], \mathbf{x}[n+1], h), \quad \forall n \in [0, N-1] \\ & \theta[0] = \gamma - \alpha, \\ & \theta[N] = \gamma + \alpha, \\ & \dot{\theta}[0] = \dot{\theta}[N]\cos(2\alpha) \\ & h_{min} \leq h \leq h_{max}.\end{aligned}$$

Although it is likely not needed for this simple example (since the dynamics are sufficiently limited), for completeness one should also add constraints to ensure that none of the intermediate points are in contact,

$$\gamma - \alpha \leq \theta[n] < \gamma + \alpha, \quad \forall n \in [1, N-1].$$

The result is a simple and clean numerical algorithm for finding the rolling limit cycle solution of the rimless wheel. Please take it for a spin:

The specific case of the rimless wheel is quite clean. But before we apply it to the compass gait, the kneed compass gait, the spring-loaded inverted pendulum, etc, then we should stop and figure out a more general form.

17.1 (Autonomous) Hybrid Systems

Recall how we modeled the dynamics of the simple legged robots. First, we derived the equations of motion (independently) for each possible contact configuration -- for example, in the spring-loaded inverted pendulum (SLIP) model we had one set of equations governing the (x, y) position of the mass during the flight phase, and a completely separate set of equations written in polar coordinates, (r, θ), describing the stance phase. Then we did a little additional work to describe the transitions between these models -- e.g., in SLIP we transitioned from flight to stance when the foot first touches the ground. When simulating this model, it means that we have a discrete "event" which occurs at the moment of foot collision, and an immediate discontinuous change to the state of the robot (in this case we even change out the state variables).

The language of *hybrid systems* gives us a rich language for describing systems of this form, and a suite of tools for analyzing and controlling them. The term "hybrid systems" is a bit overloaded, here we use "hybrid" to mean both discrete- and continuous-time, and the particular systems we consider here are sometimes called *autonomous* hybrid systems because the internal dynamics can cause the discrete changes without any exogeneous input[†]. In the hybrid systems formulation, we describe a system by a set of *modes* each described by (ideally smooth) continuous dynamics, a set of *guards* which here are continuous functions whose zero-level set

[†]This is in contrast to, for instance, the mode of a power-train when

describes the conditions which trigger an event, and a set of *resets* which describe the discrete update to the state that is triggered by the guard. Each guard is associated with a particular mode, and we can have multiple guards per mode. Every guard has at most one reset. You will occasionally hear guards referred to as "witness functions", since they play that role in simulation, and resets are sometimes referred to as "transition functions".

The imagery that I like to keep in my head for hybrid systems is illustrated below for a simple example of a robot's heel striking the ground. A solution trajectory of the hybrid system has a continuous trajectory inside each mode, punctuated by discrete updates when the trajectory hits the zero-level set of the guard (here the distance between the heel and the ground becomes zero), with the reset describing the discrete change in the state variables.

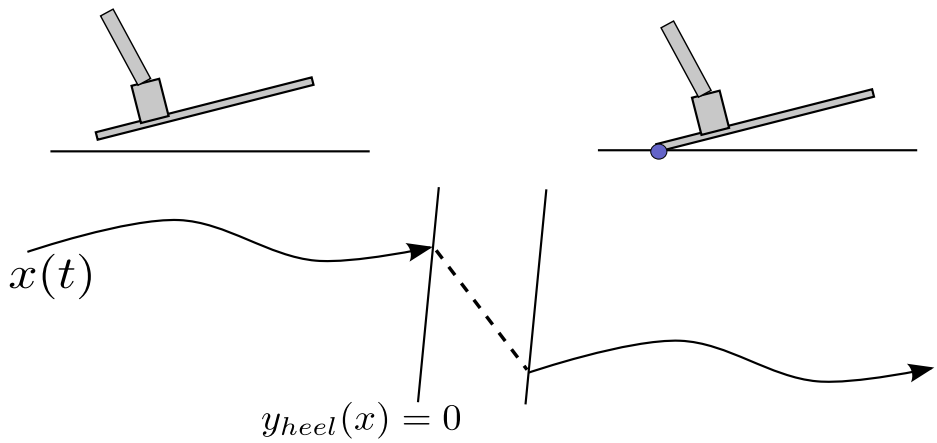

Figure 17.2 - Modeling contact as a hybrid system.

For this robot foot, we can decompose the dynamics into distinct modes: (1) foot in the air, (2) only heel on the ground, (3) heel and toe on the ground, (4) only toe on the ground (push-off). More generally, we will write the dynamics of mode i as $\mathbf{f}_i$, the guard which signals the transition mode i to mode j as $\phi_{i,j}$ (where $\phi_{i,j}(\mathbf{x}_i) > 0$ inside mode i), and the reset map from i to j as $\mathbf{\Delta}_{i,j}$, as illustrated in the following figure:

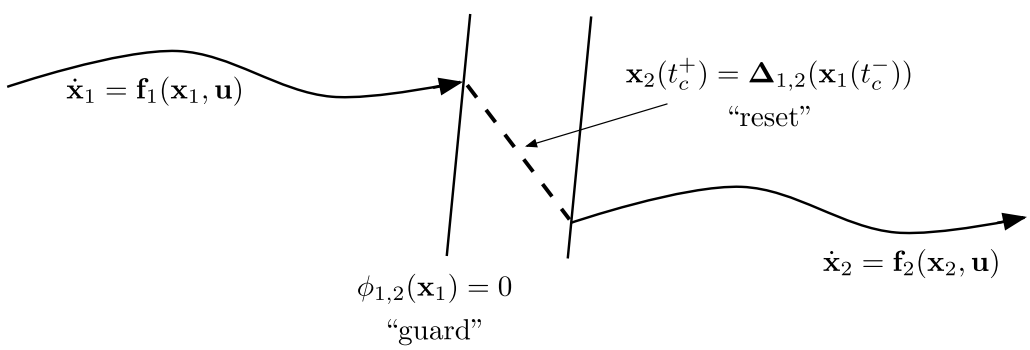

Figure 17.3 - The language of hybrid systems: modes, guards, and reset maps.

17.1.1 Hybrid trajectory optimization

Using the more general language of modes, guards, and resets, we can begin to formulate the "hybrid trajectory optimization" problem. In hybrid trajectory optimization, there is a major distinction between trajectory optimization where *the mode sequence is known apriori* and the optimization is just attempting to solve for the continuous-time trajectories, vs one in which we must also discover the mode sequence.

Given a fixed mode sequence

For the case when the mode sequence is fixed, then hybrid trajectory optimization can be as simple as stitching together multiple individual mathematical programs into a single mathematical program, with the boundary conditions constrained to enforce the guard/reset constraints. Using the shorthand $\mathbf{x}_k$ for the state in the kth segment of the sequence, and m_k for the mode in segment k, we can write:

$$\begin{aligned}
\text{find}_{\mathbf{x}_k[\cdot],h_k} \quad \text{subject to} \quad & \mathbf{x}_0[0] = \mathbf{x}_0, \\
\forall k, \forall n_k \in [0, N_k - 1], \quad & \text{collocation constraints}_{m_k}(\mathbf{x}_k[n_k], \mathbf{x}_k[n_k+1], h_k[n_k]), \\
& h_{min} \le h_k[n_k] \le h_{max}, \\
\forall k \in [0, K-1] \quad & \phi_{m_k,m_{k+1}}(\mathbf{x}_k[N_k]) = 0, \\
& \mathbf{x}_{k+1}[0] = \mathbf{\Delta}_{m_k,m_{k+1}}(\mathbf{x}_k[N_k]), \\
\forall k, n_k, m \in G \quad & \phi_{m_k,m}(\mathbf{x}_k[n_k]) > 0.
\end{aligned}$$

The constraints in the last line ensure that no trajectories intersect an *unscheduled* guard, and G is taken to be the tuples of k, n_k, m which list all guards and times *except* the scheduled ones. It is then natural to add control inputs (as additional decision variables), and to add an objective and any more constraints.

Example 17.2 (A basketball trick shot.)

As a simple example of this hybrid trajectory optimization, I thought it would be fun to see if we can formulate the search for initial conditions that optimizes a basketball "trick shot". A quick search turned up this video for inspiration.

Let's start simpler -- with just a "bounce pass". We can capture the dynamics of a bouncing ball (in the plane, ignoring spin) with some very simple dynamics:

$$\mathbf{q} = \begin{bmatrix} x \\ z \end{bmatrix}, \qquad \ddot{\mathbf{q}} = \begin{bmatrix} 0 \\ -g \end{bmatrix}.$$

During any time interval without contact of duration h, we can actually integrate these dynamics perfectly:

$$\mathbf{x}(t+h) = \begin{bmatrix} x(t) + h\dot{x}(t) \\ z(t) + h\dot{z}(t) - \frac{1}{2}gh^2 \\ \dot{x}(t) \\ \dot{z}(t) - hg \end{bmatrix}.$$

With the bounce pass, we just consider collisions with the ground, so we have a guard, $\phi(\mathbf{x}) = z$, which triggers when $z = 0$, and a reset map which assumes an elastic collision with coefficient of restitution e:

$$\mathbf{x}^+ = \mathbf{\Delta}(\mathbf{x}^-) = \begin{bmatrix} x^- & z^- & \dot{x}^- & -e\dot{z}^- \end{bmatrix}^T.$$

We'll formulate the problem as this: given an initial ball position $(x = 0, z = 1)$, a final ball position 4m away $(x = 4, z = 1)$, find the initial velocity to achieve that goal in 5 seconds. Clearly, this implies that $\dot{x}(0) = 4/5$. The interesting question is -- what should we do with $\dot{z}(0)$? There are multiple solutions -- which involve bouncing a different number of times. We can find them all with a simple hybrid trajectory optimization, using the observation that there are two possible solutions for each number of bounces -- one that starts with a positive $\dot{z}(0)$ and one with a negative $\dot{z}(0)$.

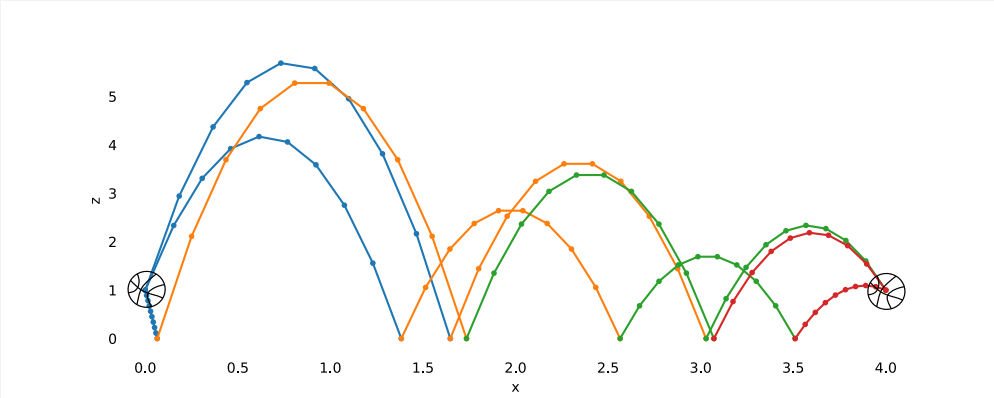

Figure 17.4 - Trajectory optimization to find solutions for a "bounce pass". I've plotted all solutions that were found for 2, 3, or 4 bounces... but I think it's best to stick to a single bounce if you're using this on the court.

Now let's try our trick shot. I'll move our goal to $x_f = -1m, z_f = 3m$, and introduce a vertical wall at $x = 0$, and move our initial conditions back to $x_0 = -.25m$. The collision dynamics, which now must take into account the spin of the ball, are in the appendix. The first bounce is against the wall, the second is against the floor. I'll also constrain the final velocity to be down (have to approach the hoop from above). Try it out.

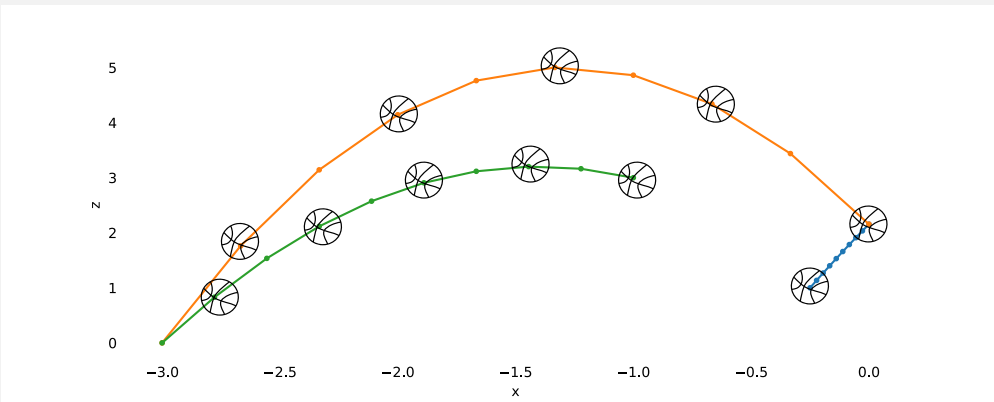

Figure 17.5 - Trajectory optimization for the "trick shot". Nothing but the bottom of the net! The crowd is going wild!

In this example, we could integrate the dynamics in each segment analytically. That is the exception, not the rule. But you can take the same steps with a little more code to use, e.g. direct transcription or collocation with multiple break points in each segment.

Direct shooting

It is also possible to optimize the trajectory in a single shot by taking gradients through the guard/reset map. This method does not require an explicit mode sequence, but is prone to having local minima (since there is no gradient to "pull" the trajectory towards a guard that is not visited along the nominal trajectory).

In order to achieve this, we need to take the gradient of the cost with respect to the trajectory parameters. The methods we've developed previously for calculating gradients using the adjoint method (e.g. "backpropagation through time") in discrete and continuous time can be combined to handle the hybrid case. The derivation is *almost* as simple as using the continuous adjoint equation to solve (from the final time forward) for the continuous modes, and the discrete adjoint equation to handle the jump discontinuity. The only detail is that we also have to consider variations

relative to the contact time; this gradient is often referred to as the "Saltation matrix".

Consider the case of two modes and one transition in our canonical hybrid system picture above. Then one could write the total cost as

$$J_\alpha = \int_0^{t_c^-} \ell_1(\mathbf{x}_1, \mathbf{u}_\alpha) dt + \int_{t_c^+}^{t_f} \ell_2(\mathbf{x}_2, \mathbf{u}_\alpha) dt,$$
$$\dot{\mathbf{x}}_1 = \mathbf{f}_1(\mathbf{x}_1, \mathbf{u}_\alpha), \quad \dot{\mathbf{x}}_2 = \mathbf{f}_2(\mathbf{x}_2, \mathbf{u}_\alpha), \quad \mathbf{x}_2(t_c^+) = \mathbf{\Delta}_{1,2}(\mathbf{x}_1(t_c^-)).$$

We wish to take the gradient of J with respect to the trajectory parameters, α. The gradient of the first and second integrals is exactly the familiar continuous-time case, but the second integral must be initialized with $\frac{\partial \mathbf{x}_2(t_c^+)}{\partial \alpha_i}$, which should be related to $\frac{\partial \mathbf{x}_1(t_c^-)}{\partial \alpha_i}$ via the discrete transition. The important point is to capture the dependence of this transition not only on $\mathbf{x}$ but also on t_c, which both depend on α. Writing it out we have...

Deriving hybrid models: minimal vs floating-base coordinates

There is some work to do in order to derive the equations of motion in this form. Do you remember how we did it for the rimless wheel and compass gait examples? In both cases we assumed that exactly one foot was attached to the ground and that it would not slip, this allowed us to write the Lagrangian as if there was a pin joint attaching the foot to the ground to obtain the equations of motion. For the SLIP model, we derived the flight phase and stance phase using separate Lagrangian equations each with different state representations. I would describe this as the *minimal coordinates* modeling approach -- it is elegant and has some important computational advantages that we will come to appreciate in the algorithms below. But it's a lot of work! For instance, if we also wanted to consider friction in the foot contact of the rimless wheel, we would have to derive yet another set of equations to describe the sliding mode (adding, for instance, a prismatic joint that moved the foot along the ramp), plus the guards which compute the contact force for a given state and the distance from the boundary of the friction cone, and on and on.

Fortunately, there is an alternative modeling approach for deriving the modes, guards, and resets for contact that is more general (though it makes the dynamics of each mode a bit more complex). We can instead model the robot in the *floating-base coordinates* -- we add a fictitious six degree-of-freedom "floating-base" joint connecting some part of the robot to the world (in planar models, we use just three degrees-of-freedom, e.g. (x, z, θ)). We can derive the equations of motion for the floating-base robot once, without considering contact, then add the additional constraints that come from being in contact as contact forces which get applied to the bodies. The resulting manipulator equations take the form

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \sum_i \mathbf{J}_i^T(\mathbf{q})\lambda_i, \tag{1}$$

where λ_i are the constraint forces and $\mathbf{J}_i$ are the constraint Jacobians. Conveniently, if the guard function in our contact equations is the signed distance from contact, $\phi_i(\mathbf{q})$, then this Jacobian is simply $\mathbf{J}_i(\mathbf{q}) = \frac{\partial \phi_i}{\partial \mathbf{q}}$. I've written the basic derivations for the common cases (position constraints, velocity constraints, impact equations, etc) in the appendix. What is important to understand here is that this is an alternative formulation for the equations governing the modes, guards, and resets, but that is it no longer a minimal coordinate system -- the equations of motion are written in $2N$ state variables but the system might actually be constrained to evolve only along a lower dimensional manifold (if we write the rimless wheel equations with three configuration variables for the floating base, it still only rotates around the toe when it is in stance and is inside the friction cone). This will have implications for our algorithms.

Note that there are some subtleties about using direct collocation methods with

systems parameterized in the floating-base coordinates with additional contact constraints -- the naive implementation of direct collocation may not have enough degrees of freedom in the spline to satisfy the dynamics constraints exactly at the knot points and the collocation points. [1] shows the natural extension of direct collocation to constrained systems; and describes more completely the advantages of parameterizing contact forces as decision variables.

For completeness, you might also hear the term *maximal coordinates*. This is the extreme case where every rigid body in the robot is parameterized with its own floating-base, and even the robot joints are implemented as constraints holding the links together. This has been used by some notable simulation engines (e.g. the Open Dynamics Engine), and may have advantages for some computational methods. But for our purposes, using the dynamics in floating-base coordinates is typically much more effective.

17.1.3 Discrete control (between events)

In our discussion of the SLIP model of running robots, we introduced a surprisingly simple but effective approach to achieving a "deadbeat" controller, with control decisions happening just once per cycle. There is a natural generalization of this idea to more general hybrid systems...

17.1.4 Hybrid LQR

Coming soon (see [2]).

17.1.5 Hybrid Lyapunov analysis

Coming soon (see [3, 4]).

17.2 Contact-implicit trajectory optimization

Coming soon. (for starters, see [5], or [6] for a recent survey).

17.3 Exercises

Exercise 17.1 (Finding the compass gait limit cycle)

In this exercise we use trajectory optimization to identify a limit cycle for the compass gait. We use a rather general approach: the robot dynamics is described in floating-base coordinates and frictional contacts are accurately modeled. , you are asked to code many of the constraints this optimization problem requires:

a. Enforce the contact between the stance foot and the ground at all the break points.
b. Enforce the contact between the swing foot and the ground at the initial time.
c. Prevent the penetration of the swing foot in the ground at all the break points. (In this analysis, we will neglect the scuffing between the swing foot and the ground which arises when the swing leg passes the stance leg.)
d. Ensure that the contact force at the stance foot lies in the friction cone at all the break points.
e. Ensure that the impulse generated by the collision of the swing foot with the ground lies in the friction cone.

References

1. Michael Posa and Scott Kuindersma and Russ Tedrake, "Optimization and stabilization of trajectories for constrained dynamical systems", *Proceedings of the International Conference on Robotics and Automation (ICRA)* , pp. 1366-1373, May, 2016. [link]

2. Ian R. Manchester and Uwe Mettin and Fumiya Iida and Russ Tedrake, "Stable Dynamic Walking Over Uneven Terrain", *The International Journal of Robotics Research (IJRR)*, vol. 30, no. 3, pp. 265-279, January 24, 2011. [link]

3. Ian R. Manchester, "Transverse Dynamics and Regions of Stability for Nonlinear Hybrid Limit Cycles", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2241 [math.OC]*, Aug-Sep, 2011.

4. Ian R. Manchester and Mark M. Tobenkin and Michael Levashov and Russ Tedrake, "Regions of Attraction for Hybrid Limit Cycles of Walking Robots", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2247 [math.OC]*, 2011.

5. Michael Posa and Cecilia Cantu and Russ Tedrake, "A Direct Method for Trajectory Optimization of Rigid Bodies Through Contact", *International Journal of Robotics Research*, vol. 33, no. 1, pp. 69-81, January, 2014. [link]

6. Patrick M Wensing and Micheal Posa and Yue Hu and Adriend Escande and Nicolas Mansard and Andrea Del Prete, "Optimization-based control for dynamic legged robots", *arXiv preprint arXiv:2211.11644*, 2022.

© Russ Tedrake, 2024